

循环神经网络姓名分类

实验报告

姓名：孙佳乐

学号：2310417

专业：计算机科学与技术

2026 年 4 月 22 日

目录

1 实验目的	2
2 实验环境	2
3 数据集与预处理	2
3.1 数据集来源	2
3.2 数据预处理	2
3.3 数据批处理	2
4 模型结构与实现	3
4.1 RNN 模型结构	3
4.2 LSTM 模型结构	3
4.3 训练配置	4
5 实验结果与分析	4
5.1 RNN 实验结果	4
5.2 LSTM 实验结果	5
5.3 对比分析	7
6 为什么 LSTM 性能优于 RNN	7
6.1 RNN 的梯度问题	7
6.2 LSTM 的门控机制	7
6.3 LSTM 的优势总结	8
7 结论	8
8 代码结构与文件说明	8
9 实验复现命令	8

1 实验目的

本次实验的主要目的包括：

1. 掌握循环神经网络（RNN）的基本原理与训练流程；
2. 使用 PyTorch 深度学习框架搭建 RNN 模型，完成姓名语言分类任务；
3. 实现长短时记忆网络（LSTM）并在同一任务上进行训练与性能对比；
4. 深入分析 LSTM 相比普通 RNN 的性能优势及其内在原因。

2 实验环境

- 操作系统：Windows 11
- Python 版本：3.8
- 深度学习框架：PyTorch 2.2.0
- 主要依赖库：matplotlib, numpy, torch
- 硬件加速：CUDA (NVIDIA GPU)

3 数据集与预处理

3.1 数据集来源

实验采用 PyTorch 官方教程提供的姓名数据集，包含 18 种语言的姓名文本文件。每个文件以语言名称命名（如 `English.txt`、`Chinese.txt`），每行包含一个姓名。数据集中共有 20074 个姓名样本。

3.2 数据预处理

数据预处理流程如下：

1. **Unicode 转 ASCII**: 将包含重音符号的字符(如 “Ślusàrski”)转换为纯 ASCII 字符(如 “Slusarski”), 以简化输入特征空间。
2. **字符编码**: 构建允许字符集合 (大小写字母、空格、标点符号), 为每个字符分配唯一索引 (0 预留为填充符)。
3. **序列化**: 将每个姓名转换为对应的整数索引序列。
4. **数据集划分**: 按 85%/15% 的比例划分训练集和验证集, 使用固定随机种子 (2026) 保证实验可复现。

3.3 数据批处理

由于姓名序列长度不一, 采用 `pack_padded_sequence` 技术进行动态批处理, 避免在填充位置进行无效计算, 提高训练效率。

4 模型结构与实现

4.1 RNN 模型结构

RNN 模型采用标准的循环神经网络架构，具体结构如下：

RNN 模型结构

```

1 RNNClassifier(
2     (embed): Embedding(59, 64, padding_idx=0)
3     (rnn): RNN(64, 128, batch_first=True)
4     (classifier): Linear(in_features=128, out_features=18, bias=True)
5     (log_softmax): LogSoftmax(dim=1)
6 )

```

模型参数量：30,866

RNN 的核心计算公式为：

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (1)$$

其中 h_t 为当前时刻的隐藏状态， x_t 为当前输入字符的嵌入向量。

4.2 LSTM 模型结构

LSTM 模型采用长短时记忆网络架构，结构如下：

LSTM 模型结构

```

1 LSTMClassifier(
2     (embed): Embedding(59, 64, padding_idx=0)
3     (lstm): LSTM(64, 128, batch_first=True)
4     (classifier): Linear(in_features=128, out_features=18, bias=True)
5     (log_softmax): LogSoftmax(dim=1)
6 )

```

模型参数量：105,362

LSTM 的核心门控机制如下：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{遗忘门}) \quad (2)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{输入门}) \quad (3)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (4)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (5)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{输出门}) \quad (6)$$

$$h_t = o_t \odot \tanh(C_t) \quad (7)$$

4.3 训练配置

超参数	取值
训练轮数 (Epochs)	20
批量大小 (Batch Size)	256
学习率 (Learning Rate)	0.002
优化器	Adam
损失函数	NLLLoss
梯度裁剪阈值	3.0
嵌入维度	64
隐藏层维度	128

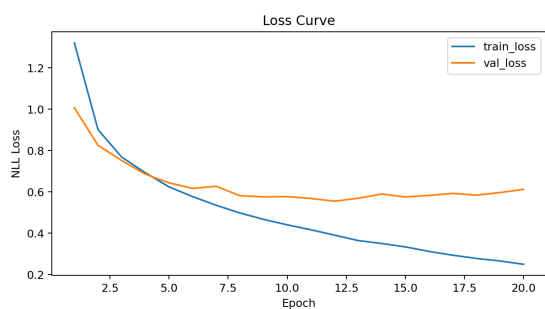
表 1: 训练超参数配置

5 实验结果与分析

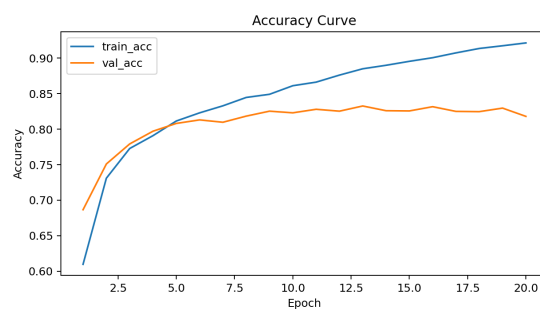
5.1 RNN 实验结果

RNN 模型在验证集上的表现：

- 验证集损失 (val_loss): 0.5691
- 验证集准确率 (val_acc): 0.8327



(a) RNN 损失曲线



(b) RNN 准确率曲线

图 1: RNN 训练曲线

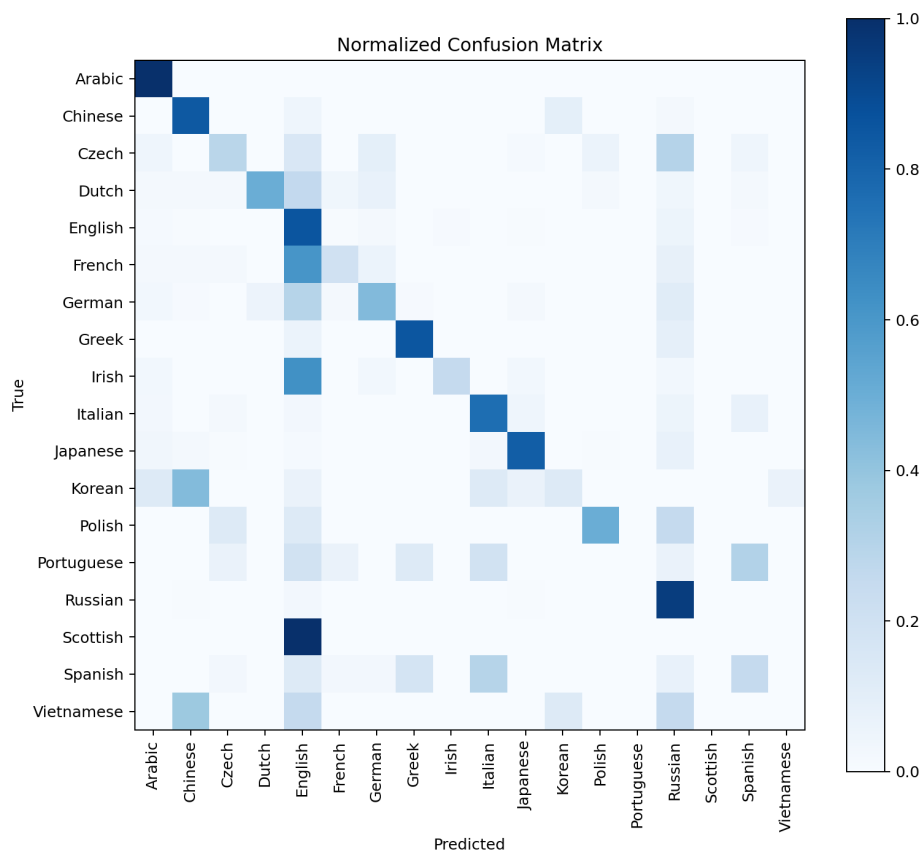


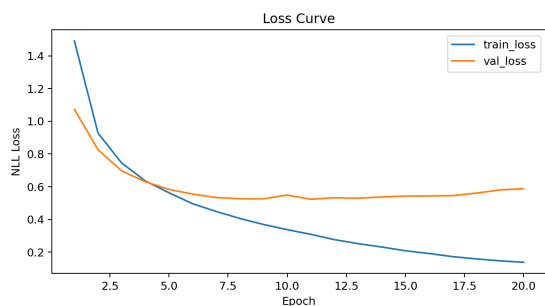
图 2: RNN 归一化混淆矩阵

从混淆矩阵可以看出, RNN 模型在大部分语言类别上表现良好, 但在某些相似语言 (如 Czech 与 Polish) 之间存在一定的混淆。

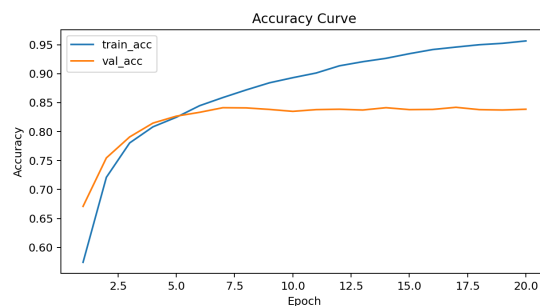
5.2 LSTM 实验结果

LSTM 模型在验证集上的表现:

- 验证集损失 (val_loss): 0.5452
- 验证集准确率 (val_acc): 0.8420



(a) LSTM 损失曲线



(b) LSTM 准确率曲线

图 3: LSTM 训练曲线

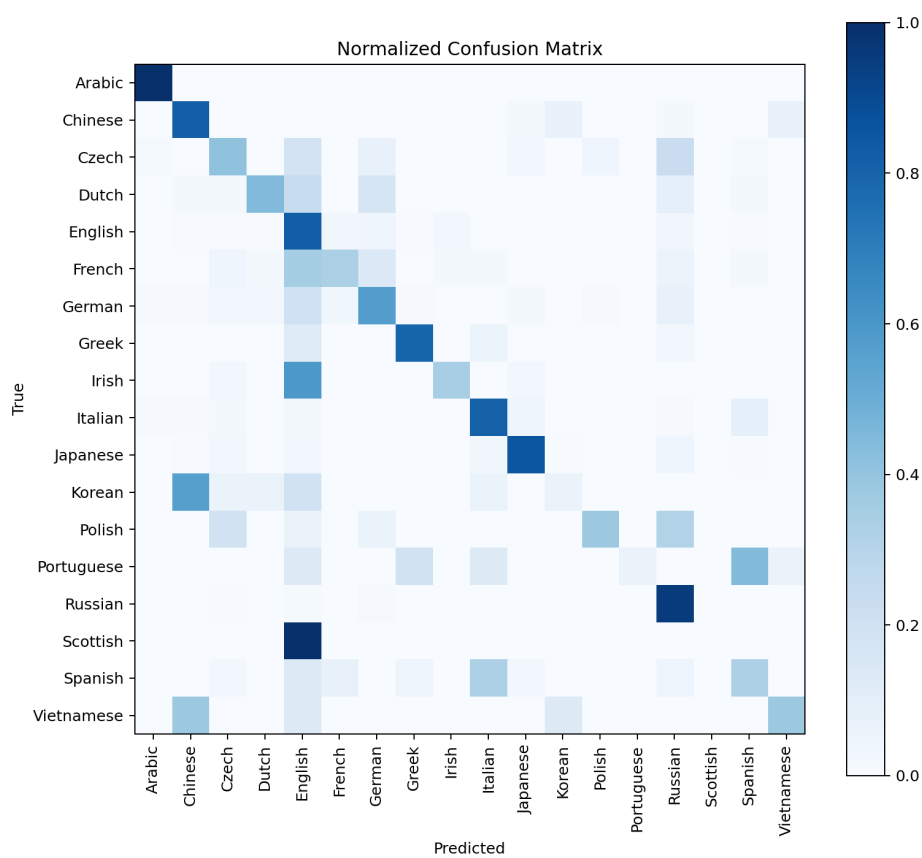


图 4: LSTM 归一化混淆矩阵

5.3 对比分析

模型	验证集损失	验证集准确率
RNN	0.5691	0.8327
LSTM	0.5452	0.8420
提升	-0.0239	+0.0093 (+0.93%)

表 2: RNN 与 LSTM 性能对比

从实验结果可以看出, LSTM 在验证集准确率上相比 RNN 提升了约 0.93%, 同时损失值更低, 表明 LSTM 具有更好的拟合能力和泛化性能。

6 为什么 LSTM 性能优于 RNN

6.1 RNN 的梯度问题

普通 RNN 的隐藏状态递推公式为:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (8)$$

在反向传播过程中, 梯度需要沿着时间步反复乘以权重矩阵 W_{hh} 。对于长度为 T 的序列, 梯度传播路径中的雅可比矩阵乘积为:

$$\frac{\partial h_T}{\partial h_0} = \prod_{t=1}^T \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=1}^T \text{diag}(\tanh'(h_t)) \cdot W_{hh}^\top \quad (9)$$

当 T 较大时, 若 W_{hh} 的特征值小于 1, 梯度会指数级衰减 (梯度消失); 若特征值大于 1, 梯度会指数级爆炸 (梯度爆炸)。这导致 RNN 难以学习长距离的依赖关系。

6.2 LSTM 的门控机制

LSTM 通过引入门控机制和独立的细胞状态 (Cell State) 有效缓解了梯度问题:

1. **遗忘门**: 控制历史信息的保留程度, 允许模型选择性地遗忘不重要的信息。

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (10)$$

2. **输入门**: 控制新信息的写入程度, 避免无关信息的干扰。

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (11)$$

3. **细胞状态更新**: 细胞状态通过加法运算传递梯度, 梯度传播路径更稳定。

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (12)$$

4. **输出门**: 控制细胞状态向隐藏状态的输出。

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t) \quad (13)$$

6.3 LSTM 的优势总结

1. **梯度稳定性**: 细胞状态的更新以加法为主, 梯度可直接沿 $C_t = C_{t-1} + \dots$ 路径传播, 有效缓解梯度消失问题。
2. **长时记忆能力**: 门控机制使 LSTM 能够选择性地保留长期信息, 对于姓名分类这类需要捕捉字符序列整体模式的任务尤为重要。
3. **灵活的上下文建模**: LSTM 能够根据当前输入动态调整信息流动, 对不同语言姓名的字符组合模式有更强的表达能力。
4. **更快的收敛速度**: 从训练曲线可以看出, LSTM 的损失下降更为平滑, 收敛速度更快。

在本次姓名分类任务中, LSTM 的优势体现在: 能够更好地捕捉不同语言姓名的构词规律 (如后缀特征、音节模式等), 从而在验证集上取得更高的分类准确率。

7 结论

本次实验成功实现了基于 RNN 和 LSTM 的姓名语言分类系统。通过对比分析得出以下结论:

1. 两种模型均能有效完成姓名语言分类任务, 验证准确率均超过 83%。
2. LSTM 凭借其门控机制和独立的细胞状态设计, 在长序列建模方面优于普通 RNN, 验证准确率提升约 0.93%。
3. LSTM 的训练过程更加稳定, 损失曲线更平滑, 体现了其对梯度消失问题的有效缓解。
4. 实验结果验证了理论分析: LSTM 的门控结构确实能够更好地捕捉字符序列中的长距离依赖关系。

8 代码结构与文件说明

- `run_lab2.py`: 实验主脚本, 包含数据加载、模型定义、训练与评估的完整流程。
- `outputs/rnn/`: RNN 模型的输出目录, 包含训练曲线、混淆矩阵、模型权重等。
- `outputs/lstm/`: LSTM 模型的输出目录, 包含训练曲线、混淆矩阵、模型权重等。
- `outputs/comparison_summary.json`: 两模型性能对比的汇总文件。

9 实验复现命令

在项目目录下执行以下命令可复现完整实验:

```
1 python run_lab2.py --data-root data --output outputs \  
2   --epochs 20 --batch-size 256 --hidden-size 128 \  
3   --emb-dim 64 --lr 0.002
```